Generics

Section Overview

- Before Java 5, collections stored Object, which caused two problems:

```
> List list = new ArrayList();
```

- list.add("Hello");
- list.add(10); // compiler allows mixing types
- String s = (String) list.get(1); // ClassCastException at RUNTIME

Section Overview

- Generics let you write code that works with a type supplied later, while the compiler checks types at compile time:

```
> List<String> list = new ArrayList<>();
```

- list.add("Hello");
- list.add(10); // compile-time error
- String s = list.get(0); // no cast needed

Generic class

```
> class Box<T> { private T item; public Box(T item) { this.item = item; } public T get() { return item; } public void set(T item) { this.item = item; } } public class Demo { public static void main(String[] args) { Box<Integer> b1 = new Box<>(25); // T = Integer Box<String> b2 = new Box<>("Java"); // T = String int x = b1.get(); // no cast String s = b2.get(); // b1.set("text"); // compile error: String is not Integer } }
```

Section Overview

Multiple type parameters

```
> class Pair<K, V> { private K key; private V value; public Pair(K key, V value) { this.key = key; this.value = value; } public K getKey() { return key; } public V getValue() { return value; } @Override public String toString() { return key + " -> " + value; } }
```

```
> public class Main { public static void main(String[] args) { Pair<String, Integer> p1 = new Pair<>("Anu", 95); Pair<Integer, String> p2 = new Pair<>(101, "Chennai"); System.out.println(p1); System.out.println(p2); System.out.println("Name : " + p1.getKey()); System.out.println("Marks : " + p1.getValue()); } }
```

Section Overview

- Pair<K, V> is a generic class with two type parameters: K for the key and V for the value.

Pair<String, Integer> makes K = String and V = Integer, so getKey() returns a String and getValue() returns an Integer. No casting is needed.

- Pair<Integer, String> reuses the same class with the types reversed.
- println(p1) calls the overridden toString(), which prints key -> value.

Section Overview

- Type arguments must be reference types. Box<int> is illegal; use Box<Integer>. Autoboxing handles the conversion.
- The diamond operator <> (Java 7+) lets the compiler infer the type on the right side.

Generic interface

- An interface that declares type parameters. Implementing classes have two choices.
- interface InterfaceName<T> {

```
> // abstract methods using T
```

- void method1(T data);
- T method2();
- }

Example Program using Interface

```
> interface Printer<T> { void print(T data); } class IntegerPrinter implements Printer<Integer> { public void print(Integer data) { System.out.println("Integer value: " + data); } } class StringPrinter implements Printer<String> { public void print(String data) { System.out.println("String value: " + data); } }
```

```
> public class Main { public static void main(String[] args) { Printer<Integer> ip = new IntegerPrinter(); Printer<String> sp = new StringPrinter(); ip.print(100); sp.print("Hello Generics"); } }
```

Generic Methods

Note: If we remove the first<Type>in the above method, we will get compilation error (cannot find symbol 'Type') since it represents the declaration of the symbol.

- In many cases the user of the method need not indicate the type parameters, as they can be inferred:
- Entry<String, String> pair = Entry.twice("Hello");
- The parameters can be explicitly added if needed:
- Entry<String, String> pair = Entry.<String>twice("Hello");

```
public static <Type> Entry<Type, Type> twice(Type value) {
    return new Entry<Type, Type>(value, value);
}
```

Generic methods

```
> public class Main {   // Generic method: works with any type T   public static <T> void printArray(T[] array) {     for (T item : array) {       System.out.println(item);     }   }   public static void main(String[] args) {     // Array of Strings     String[] names = {"Jenny", "Liam"};     // Array of Integers     Integer[] numbers = {1, 2, 3};     // Call the generic method with both arrays     printArray(names);     printArray(numbers);   } }
```

Bounded type parameters

An unrestricted T is treated as Object, so you can only call Object methods on it. Bounds restrict what T can be, and in exchange let you call the bound's methods.

Section Overview

- You can use the extends keyword to limit the types a generic class or method can accept.
- For example, you can require that the type must be a subclass of Number:

Section Overview

```
> class Stats<T extends Number> {   T[] nums;   // Constructor   Stats(T[] nums) {     this.nums = nums;   }   // Calculate average   double average() {     double sum = 0;     for (T num : nums) {       sum += num.doubleValue();     }     return sum / nums.length;   } }
```

Section Overview

```
> public class Main {   public static void main(String[] args) {     // Use with Integer     Integer[] intNums = {10, 20, 30, 40};     Stats<Integer> intStats = new Stats<>(intNums);     System.out.println("Integer average: " + intStats.average());     // Use with Double     Double[] doubleNums = {1.5, 2.5, 3.5};     Stats<Double> doubleStats = new Stats<>(doubleNums);     System.out.println("Double average: " + doubleStats.average());   } }
```

Section Overview

- <T extends Number>: Restricts T to only work with numeric types like Integer, Double, or Float.
- .doubleValue(): Converts any number to a double for calculation.
- Works for any array of numbers as long as the elements are subclasses of Number.

The ? (Wildcard) in Java Generics

? means "some unknown type". It is used when you use a generic type (method parameters, variables), not when you declare a class or interface.

- We write T to declare, and ? to say "I accept any type here and don't care which."

1. Unbounded: <?>

```
> import java.util.*; public class Main { static void printAll(List<?> list) { for (Object o : list) { System.out.print(o + " "); }
System.out.println(); } public static void main(String[] args) { printAll(List.of(1, 2, 3)); printAll(List.of("A", "B"));
printAll(List.of(1.5, 2.5)); } }
```

- Output:
- 1 2 3
- A B
- 1.5 2.5

2. Upper bounded: <? extends Number>

```
> static double sum(List<? extends Number> list) { double total = 0; for (Number n : list) { total += n.doubleValue(); } return total; } sum(List.of(1, 2, 3)); // List<Integer> -> 6.0 sum(List.of(1.5, 2.5)); // List<Double> -> 4.0
```

Section Overview

- We can read elements as Number.
- We cannot add to the list (except null), because the compiler does not know the exact type:
- static void test(List<? extends Number> list) {
- Number n = list.get(0); // OK
- list.add(5); // ERROR: the list might really be List<Double>
- }

3. Lower bounded: <? super Integer>

```
> static void addNumbers(List<? super Integer> list) { list.add(10); list.add(20); } addNumbers(new ArrayList<Integer>());
addNumbers(new ArrayList<Number>()); addNumbers(new ArrayList<Object>());
```

? super Integer means "some unknown type that is Integer or any superclass of Integer". The bound sets a floor: the type can be Integer or anything above it in the hierarchy.

Section Overview

- Object
- ■■■ Number
- ■■■ Integer

Example using Comparator

- static <T> void sort(List<T> list, Comparator<? super T> c)
- Comparator<Number> byValue = (a, b) -> Double.compare(a.doubleValue(), b.doubleValue());

```
> List<Integer> nums = new ArrayList<>(List.of(5, 2, 9));
```

- nums.sort(byValue); // works because Comparator<? super Integer> accepts Comparator<Number>

```
> System.out.println(nums); // [2, 5, 9]
```

Section Overview

- A comparator that can compare any Number can certainly compare Integers, so super lets you reuse it.
- Without super, you would need a separate Comparator<Integer>.